

Market Warrior

Master Developer Handoff Pack (v3)

Prepared: 2025-12-15
Owner domain: marketwarrior.club

1. Purpose

This pack is designed so you can hand the project to a new developer without back-and-forth. It contains: specs, UI templates, day content, current code snapshots, Supabase exports, and a clear priority plan. It also explains how to reset Supabase safely and how to upload everything to GitHub in an owner-friendly format.

2. What you already have (assets included in the bundle)

Item	Details
Specs	Proposal - Market Warrior.docx; Market_Warrior_Technical_Specification_.docx; Admin panel specs.docx
UI templates (must match 1:1)	dashboard.html; admin-panel-enhanced.html (these define the exact look you want)
Course content	Day 1-30 HTML content (ZIPs included)
Current codebase snapshot	Latest developer ZIP: market-warrior-14.12.2025 05.19.zip (plus older reference zips)
Supabase exports	Table/column inventory + RLS policies CSVs (used for audit and/or rebuild)
Branding	market-warrior-logo.jpg

3. Current accounts & links (for onboarding the new developer)

- Supabase project (current):
<https://supabase.com/dashboard/project/gvpaemdpmwyncdztlczy>
- GitHub repo (current): <https://github.com/mikemitakidis/marketwarriorproject>
- Vercel project (current): <https://vercel.com/mikemitakidis-projects/marketwarriorproject>
- Stripe product (current):
https://dashboard.stripe.com/acct_1SBNaN8SNp2YoTH6/products/prod_TWtuQ72mY4csdx
- Domain/DNS: marketwarrior.club managed in Squarespace; needs DNS pointing to Vercel once ready.

- Business email: support@marketwarrior.club (Google Workspace).

Note: Never share secret key values in GitHub. Developers should receive access via account invites, or you can add secrets in Vercel Environment Variables yourself.

4. Locked decisions (avoid scope creep)

- Video platform: YouTube only (unlisted embeds).
- PayPal: optional later; not required for MVP.
- Unlock logic: Day N unlocks using SERVER TIME and requires previous day completion.
- Completion definition (recommended): quiz passed + task submitted.
- UI must match provided HTML templates visually, then be wired to secure backend logic.

5. Recommended architecture

Because you want a heavy Master Admin Panel (CMS editing + preview workflows), Next.js is strongly recommended. The existing HTML templates can be converted to Next.js pages while keeping the same exact design.

- Frontend: Next.js on Vercel (App Router).
- Backend: Vercel serverless functions / API routes for all sensitive operations.
- Data/Auth/Storage: Supabase (Postgres + Auth + Storage + RLS).
- Payments: Stripe Checkout + Stripe webhook.
- Email: Supabase Auth SMTP via Resend; Resend API key optionally used for lifecycle emails.

6. The biggest problems to fix first (why the current site feels 'illogical')

- Terms acceptance must persist: once a user agrees, it must be stored in user_profiles and checked on login; it must not prompt again unless user has never agreed.
- Unlock and completion must be server-controlled: never trust browser time, browser score, or browser 'completed' flags.
- Protected content must not be readable from Supabase directly by any authenticated user (course_content and quiz_questions must not be publicly readable).
- Payments must be webhook-confirmed: access is granted only after Stripe webhook verifies the payment is valid and expected.

7. Work plan for the new developer (priority order)

7.1 P0 — Launch blockers (security + anti-cheat)

- Server-side quiz validation (60%+): client submits answers only; server loads correct answers from DB and computes score; never ship correct answers to browser.
- Server-time unlock status endpoint: dashboard/day page render from server response only.
- Auth + entitlement required on every course endpoint: user must be logged in, paid, and day must be unlocked.
- Stripe: checkout created server-side; webhook validates signature + expected price/amount; grants access + sets challenge_start_date.
- Remove secret fallbacks: server endpoints must fail fast if SUPABASE_SERVICE_KEY missing; no secrets in client.
- DB/RLS alignment: enforce that course_content/quiz_questions are not readable from the client; users can only access their own progress/results/submissions.

7.2 P1 — MVP end-to-end (paid user completes Day 1)

- Dynamic day template loads Day X from DB content (not hardcoded objects).
- Task submission endpoint (text + optional file upload to Storage) and tracking in DB.
- Terms gate: welcome page sets agreed_to_terms=true; dashboard and APIs block until true.
- Certificate generation after Day 30 completion (protected).

7.3 P2 — Master Admin Panel (CMS + CRM)

- Admin authentication/authorization (is_admin).
- CMS: edit content_html, YouTube video ID, task steps; quiz editor; preview before publish.
- Settings: price, access duration, max devices, support email, etc.
- User CRM: search, view progress, unlock, extend access, reset devices, refund (Stripe).
- Promo codes: CRUD + expiry/max uses; validate server-side.
- Announcements/live feed bar shown on student dashboard.

7.4 P3 — Automation

- Journal lead magnet + nurture email sequence (5–7 days).
- Lifecycle emails: purchase welcome, day unlock notifications, inactivity reminders.
- Affiliate tracking and dashboard; payouts later (Stripe Connect); PayPal optional later.

8. Supabase: start fresh or keep existing?

If you do not have real customers or production data, starting with a NEW Supabase project is usually easier. A new Supabase project will generate new keys (URL/anon/service). You will update Vercel env vars.

8.1 New Supabase means new keys (answer to your question)

- Yes — new Supabase project = new SUPABASE_URL, NEXT_PUBLIC_SUPABASE_ANON_KEY, SUPABASE_SERVICE_KEY.
- Google Workspace connection: if you configured SMTP/OAuth inside Supabase Auth, you will reconfigure it in the new project; your Workspace email itself remains the same.
- Resend: you will keep the same RESEND_API_KEY, but you may need to re-verify your domain and re-enter SMTP settings in Supabase Auth.

8.2 Recommended reset sequence (high level)

1. Create new Supabase project.
2. Run the baseline SQL migration included in the bundle: supabase/migrations/001_init.sql (developer can refine).
3. Create Storage bucket(s): task-uploads (private).
4. Supabase Auth settings: set SITE URL and Redirect URLs to your Vercel/custom domain; configure SMTP (Resend).
5. Update Vercel Environment Variables with the new Supabase keys and redeploy.
6. Run the acceptance tests checklist (section 12).

9. Vercel Environment Variables checklist

- NEXT_PUBLIC_APP_URL
- NEXT_PUBLIC_SUPABASE_URL
- NEXT_PUBLIC_SUPABASE_ANON_KEY
- SUPABASE_SERVICE_KEY (server-only)
- STRIPE_SECRET_KEY (server-only)
- STRIPE_WEBHOOK_SECRET (server-only)
- RESEND_API_KEY (server-only; for app emails; Supabase Auth SMTP configured separately)

10. Domain (Squarespace) -> Vercel

7. In Vercel, add 'marketwarrior.club' as a custom domain to the project.
8. Vercel will show DNS records (A/CNAME).
9. In Squarespace Domains DNS settings, add the records exactly as Vercel provides.

10. Wait for verification and set marketwarrior.club as Primary.
11. Update Supabase Auth 'Site URL' + 'Redirect URLs' to use the final domain.

11. GitHub upload format (owner-friendly)

If you upload everything to GitHub, the new developer can view everything instantly and start working without asking you to re-send files.

11.1 Recommended repo folder structure

- docs/
 - specs/ (proposal + technical spec + admin specs PDFs/DOCX)
 - handoff/ (this handoff pack PDF/DOCX + checklists)
- assets/
 - branding/ (logo, favicon, images)
- templates/
 - html/ (dashboard/admin templates + any other layout templates)
- content/
 - days-html/ (Day 1-30 HTML, unmodified original content)
 - days-notes/ (optional notes about missing video IDs, quiz count, etc)
- supabase/
 - exports/ (your current CSV exports)
 - migrations/ (SQL migrations used to create the fresh schema)
- archive/
 - zips/ (old code snapshots and day zips for reference only)

11.2 Upload steps (no coding)

12. Create folders in GitHub (use a .keep file to create empty folders).
13. Upload docs/templates/content/exports into their folders.
14. Keep code as code (preferred). If you cannot, upload ZIP snapshots under archive/zips/.
15. Never upload secrets; keep only .env.example placeholders.

12. Acceptance tests (Definition of Done)

- Terms acceptance persists across logout/login.
- Dashboard unlock is server-time based; changing device clock does not unlock.
- Day API never exposes correct answers; quiz score/pass computed server-side; cannot be faked.
- Task submission recorded server-side; completion requires quiz passed + task submitted.

- Stripe webhook verifies signature and expected payment; access granted only after valid event.
- RLS prevents reading course_content/quiz_questions directly from client; users only see their own rows.
- Admin panel is protected; edits + preview work; pricing changes update checkout.

Appendix A — Current Supabase table inventory (from exports)

- activity_log
- announcements
- challenge_progress
- course_content
- journal_leads
- payments
- promo_codes
- quiz_questions
- quiz_results
- referrals
- site_settings
- task_submissions
- trading_journal
- user_profiles

Appendix B — Current RLS snapshot highlights (from exports)

- RLS enabled on all public tables, not forced.
- course_content and quiz_questions currently readable by authenticated users (qual=true)
— must be removed.
- Prefer using server endpoints with service role key for course content and quiz answer validation.